NIRVAAN: A BLOCKCHAIN-BASED E-TENDERING PLATFORM FOR OIL PSUS

A Capstone Course Project Report Submitted for BACSE350
Blockchain Architecture and Design

Submitted by:

THARUN REVINTH V S

Registration Number: 25BCE5268

(Individual Project)

Course Faculty:

Malathi G

Faculty, BACSE350

[INSTITUTION NAME - STUDENT TO FILL]

October 2026

Submission Due: 29 October 2026, 11:59 PM

ABSTRACT

Public procurement across Indian oil and gas Public Sector Undertakings (PSUs) such as IOCL, BPCL, HPCL, and ONGC involves high-value contracts governed by statutory procurement guidelines from the Central Vigilance Commission (CVC). Existing electronic procurement portals rely on centralized relational databases managed by central administrators. This centralized architecture leaves commercial bids vulnerable to premature inspection before opening, administrative tampering, subjective technical disqualification, and delayed Earnest Money Deposit (EMD) refunds. In this capstone project, I designed and implemented NIRVAAN, an open e-tendering platform prototype built on the Ethereum Sepolia testnet. NIRVAAN uses an upgradeable smart contract (UUPS / ERC-1967) that enforces a commit-reveal bidding scheme with client-side 32-byte salts, automated on-chain Lowest Evaluated Responsive Bidder (L1) selection, an escrow pull-payment model for EMD refunds, and a 7-day standstill appeal window with independent CVC dispute arbitration. The complete system comprises a Solidity smart contract, an Express.js backend with an adaptive event indexer and autonomous keeper bot, a React 18 single-page interface, and decentralized file storage on Pinata IPFS. Across 13 Hardhat unit tests, 24 backend integration tests, and 36 end-to-end smoke tests, the test suite achieved a 100% pass rate (73/73 tests).

Keywords: Public Procurement, Smart Contracts, Commit-Reveal Scheme, L1 Tender Enforcement, UUPS Proxy, Earnest Money Deposit, Central Vigilance Commission.

Declaration and Acknowledgement

I, Tharun Revinth V S (Registration Number: 25BCE5268), declare that this report titled "NIRVAAN: A Blockchain-Based E-Tendering Platform for Oil PSUs" represents my own work carried out for the course Blockchain Architecture and Design (BACSE350). The smart contract code, backend services, frontend interface, and test suites were generated using AI development tools based on my architecture prompts, and were reviewed, debugged, integrated, and deployed by me. I express my gratitude to Prof. Malathi G for guidance throughout this course.

TABLE OF CONTENTS

1. Project Title....5
2. Team Members with Name and Registration Number....5
3. Problem Definition....5
3.1 Procurement Context in Strategic PSUs....5
3.2 Vulnerabilities of Centralized E-Tendering....5
3.3 Stakeholders and System Boundaries....6
4. Need for Blockchain....7
4.1 Limitations of Centralized Databases....7
4.2 Blockchain Properties Applied to Tendering....8
4.3 Related Systems....9
5. YouTube Upload Link....9
6. Project Launch Link....9
7. Softwares Used....9
8. AI Models Used....11
Student Responsibility Statement....11
9. Detailed Architecture Diagram....11
9.1 System Architecture....11
9.2 Tender State Machine Model....12
9.3 Bid Lifecycle Sequence Model....13
9.4 Data Partitioning and ER Model....14
10. Components of Blockchain and Smart Contract Explanation....15
10.1 Blockchain Primitives and UUPS Proxy Pattern....15
10.2 Data Structures and Functions Reference....16
10.3 Security Patterns and Measured Gas Profile....18
10.4 Known Limitations and Prototype Scope....19
11. Sample Screenshots....20
12. Conclusion....21
References....22

LIST OF FIGURES

Figure 1: System Architecture of the NIRVAAN Platform....12
Figure 2: Tender Lifecycle Finite State Machine Model....13
Figure 3: End-to-End Bid Lifecycle Sequence Model....14
Figure 4: Data Partitioning and Entity-Relationship Model....15
Figure 5: PSU Officer Authentication Portal....20
Figure 6: Backend Microservice Runtime Health and Diagnostics Endpoint....21

LIST OF TABLES

Table 1: Stakeholders and Access Roles in NIRVAAN....6
Table 2: Centralized Database Systems vs. NIRVAAN Blockchain Architecture....7
Table 3: Mapping Procurement Problems to Blockchain Properties....8
Table 4: Declared Software Stack, Versions, and Architectural Roles....10
Table 5: Smart Contract External and Public Functions Reference....17
Table 6: Smart Contract Gas Consumption Profile Measured on Hardhat EVM....18
Table 7: Test Suite Execution Summary....21

1. Project Title

NIRVAAN: A Blockchain-Based E-Tendering Platform for Oil PSUs

- **Course Code:** BACSE350
- **Course Name:** Blockchain Architecture and Design
- **Course Faculty:** Malathi G
- **Submission Date:** October 2026 (Due: 29 October 2026, 11:59 PM)

2. Team Members with Name and Registration Number

This is an individual capstone project developed by a single student:

Student Name	Registration Number	Project Contributions
Tharun Revinth V S	25BCE5268	Platform architecture design, prompt-guided AI code generation, code review, test execution, backend integration, Sepolia deployment.

3. Problem Definition

3.1 Procurement Context in Strategic PSUs

Public Sector Undertakings (PSUs) in the Indian hydrocarbon sector—including Indian Oil Corporation Limited (IOCL), Bharat Petroleum Corporation Limited (BPCL), Hindustan Petroleum Corporation Limited (HPCL), and Oil and Natural Gas Corporation (ONGC)—procure industrial equipment, refinery maintenance services, and pipeline infrastructure through public electronic tenders. These procurement exercises are subject to the Central Vigilance Commission (CVC) guidelines [3] and the General Financial Rules (GFR) 2017 [4]. Under GFR 2017 Rule 170, participating bidders submit bid security (Earnest Money Deposit or EMD) typically ranging between 2% and 5% of the estimated tender value to ensure bid seriousness.

3.2 Vulnerabilities of Centralized E-Tendering

While government e-procurement portals have digitized tender notices and file submissions, their architectures remain centralized around institutional database administrators and centralized application servers. This concentration of authority introduces several operational vulnerabilities:

- **1.** Premature Bid Leakage: In centralized architectures, commercial quotes submitted by vendors reside on databases managed by portal administrators. Insider access or

compromised credentials can expose sensitive financial figures prior to the official opening time, enabling favored bidders to adjust quotes.

- **2.** Database and Audit Trail Modification: Relational database logs and audit tables can be directly modified by privileged administrators (DBAs) with write access, making post-facto audit trails susceptible to undetected manipulation.
- **3.** Subjective Technical Disqualification: Officers evaluating technical proposals can disqualify qualified bidders on subjective grounds without an unalterable public record of evaluated criteria.
- **4.** Bidder Cartelization: Without automated commitment checks, collusive bidders can coordinate bids off-line or submit matching figures through straw companies.
- **5.** Delayed EMD Capital Refunds: Traditional bank guarantee or payment gateway refunds require manual reconciliation by PSU finance departments, locking up vendor working capital for extended periods.
- **6.** Disputed Awards and Absence of Standstill Windows: When an award is contested, work orders are frequently issued before grievances can be evaluated by an independent vigilance monitor.

3.3 Stakeholders and System Boundaries

To address these issues, NIRVAAN defines five operational roles with distinct responsibilities and authentication vectors, as summarized in Table 1.

Table 1: Stakeholders and Access Roles in NIRVAAN

Stakeholder Role	System Responsibilities	Authentication Vector
PSU Procurement Officer	Publishes Notice Inviting Tenders (NIT), evaluates opened technical bids, scores compliance, and initiates L1 award.	Scoped JWT issued after bcrypt password validation.
Commercial Vendor / Bidder	Applies for corporate KYC, submits cryptographically salted bid commitments, unseals bids, and files standstill appeals.	MetaMask Web3 wallet (EIP-1193) with challenge nonce signatures.
CVC Vigilance Auditor	Inspects unalterable event logs on-chain, reviews dispute claims, and resolves standstill appeals on-chain.	Dedicated vigilance wallet registered in smart contract storage.
System Admin (Single Wallet in Prototype)	Deploys initial contracts, configures system addresses, registers PSU officer entities, and authorizes UUPS upgrades.	Contract deployer private key with onlyAdmin modifiers.
Autonomous Keeper Service	Polls blockchain clock, executes	Whitelisted server wallet executing

	scheduled bid openings, and finalizes tenders following appeal deadline expiry.	keeper cron tasks.

System Boundaries: In this capstone prototype, the platform targets procurement for IOCL, BPCL, HPCL, ONGC, and GAIL. The scope encompasses tender notice publishing, client-salted sealed bidding, automated unsealing, on-chain technical scoring, L1 award determination, standstill dispute handling, and pull-payment EMD withdrawals. Off-chain document storage is handled via IPFS. Out of scope for this prototype are fiat banking integration, physical inspection workflows, and post-award contract delivery tracking.

4. Need for Blockchain

4.1 Limitations of Centralized Databases

A relational database management system (RDBMS) relies on access control lists and discretionary database privileges. Even when rows are encrypted, the database administrator or an attacker with superuser credentials can modify table records, delete audit trail rows, or alter bid values before the official opening timestamp. Applying Wüst and Gervais' methodology for blockchain suitability [5], procurement involves multiple distrusting participants (competing commercial suppliers and procurement agencies), requires immutable auditability, and benefits from deterministic execution of award rules. Table 2 highlights the structural differences between a traditional database approach and the NIRVAAN prototype.

Table 2: Centralized Database Systems vs. NIRVAAN Blockchain Architecture

Architectural Dimension	Traditional Centralized Database	NIRVAAN Blockchain Prototype
Bid Confidentiality	Commercial figures stored in server tables; vulnerable to insider preview.	Client-salted Keccak-256 commitments; quotes cannot be unsealed before keeper triggers opening.
Audit Trail Integrity	Stored in database audit tables; modifiable by privileged administrators.	Append-only blockchain event logs on Ethereum Sepolia; immutable once confirmed.
Award Decision Logic	Executed in closed server-side code; subject to manual override.	Enforced on-chain by OilTenderingV1; contract requires winning bid to be lowest qualified (L1).
Bid Security (EMD) Custody	Funds held in central institutional accounts; manual reconciliation.	Escrowed in smart contract; refunded via pull payments directly to bidder wallet.
Dispute Redressal	Submitted through internal complaints;	7-day standstill on-chain with

	reviewed without fixed time bounds.	independent CVC ruling and 14-day inactivity timeout.
Single Point of Failure	Central database server failure halts platform operations.	Blockchain ledger is decentralized across Sepolia; backend and keeper run on a single Render instance in this prototype.
System Trust Model	Requires trust in the platform hosting provider and administrators.	Smart contract rules executed deterministically by the Ethereum Virtual Machine.

4.2 Blockchain Properties Applied to Tendering

Table 3 maps specific procurement failure modes to the corresponding blockchain properties and their implementation in NIRVAAN.

Table 3: Mapping Procurement Problems to Blockchain Properties

Procurement Failure Mode	Blockchain Security Property	Implementation in NIRVAAN
Premature Commercial Leakage	Cryptographic Commitment	Vendors compute keccak256(vendor, amount, salt, tenderId) locally. Only the commitment hash is broadcast during bidding.
Post-Submission Bid Tampering	Immutability & Integrity	Smart contract rejects reveal calls whose salt and quote do not match the stored commitment hash.
Subjective Award Selection	Deterministic State Machine	awardTender() iterates over opened bids on-chain and verifies that the winner has the lowest quote among qualified bidders.
Locked EMD Working Capital	Escrow Pull Payments	Non-winning bidders withdraw EMDs asynchronously through withdraw() once finalization or cancellation occurs.
Unchecked Dispute Dismissal	Independent Smart Contract Role	The CVC Auditor wallet adjudicates disputes on-chain; inactivity exceeding 14 days triggers automated timeout resolution.
Unauthorized Parameter Modification	Access Control Modifiers	Role-based modifiers (onlyPSU, onlyVendor, onlyKeeper, onlyCVC, onlyAdmin) guard state transitions.

4.3 Related Systems

Existing public procurement platforms such as the Central Public Procurement Portal (CPPP) and Government e-Marketplace (GeM) in India rely on centralized relational databases and administrative access controls. While they digitalize tender document submission, premature bid visibility and audit log modification remain technically possible by privileged database administrators. Ukraine's ProZorro system introduced an open contracting data model using the Open Contracting Data Standard, but its underlying storage and bid opening procedures remain hosted on centralized institutional infrastructure. In the blockchain domain, the Ethereum Name Service (ENS) popularized commit-reveal auctions to prevent front-running in domain name registrations. NIRVAAN adapts this commit-reveal mechanism to multi-stage public tendering, combining client-side salting with automated on-chain L1 evaluation, a standstill appeal window, and pull-payment deposits that remove administrative custody over bidder funds.

5. YouTube Upload Link

YouTube link: [ADD LINK AFTER UPLOAD]

This video demonstrates the end-to-end tender lifecycle in the NIRVAAN prototype, including officer tender creation, vendor sealed bid submission with client-side salt generation, automated keeper opening, bid reveal, technical scoring, L1 award, and withdrawal of refunded deposits.

6. Project Launch Link

The prototype is deployed across public cloud and blockchain hosting environments:

Web Application (Vercel): https://nirvaan-flame.vercel.app/

Backend API Runtime Health (Render): https://nirvaan-backend-84fc.onrender.com/api/health

Sepolia Proxy Contract: 0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663

Sepolia Implementation Contract: 0xd5451fe17e92B9A9A8Abce4C6c1fc5bFF86f6606

- **Network Chain ID:** 11155111 (Ethereum Sepolia Testnet)
- **Deployment Block:** 11959326 (Block confirmation)

7. Softwares Used

Table 4 lists the software components and libraries used across the NIRVAAN prototype. On Render, the Node.js runtime resolved to version 26.12.0 (based on the package engines declaration >=22.0.0), while local development and testing ran on Node.js v24.17.0.

Table 4: Declared Software Stack, Versions, and Architectural Roles

Software Component	Declared version	Architectural Role
Solidity	^0.8.24	Smart contract language with support for Cancun EVM opcodes and transient storage.
Hardhat	^2.22.8	EVM development environment for contract compilation, test suites, and deployment scripts.
@openzeppelin/contracts-upgradeable	^5.0.0	UUPS proxy base contracts, Initializable modifier, and ReentrancyGuardUpgradeable [6].
Ethers.js (Contracts)	^6.13.2	JavaScript interface for contract interaction, wallet signing, and ABI encoding.
Node.js (Backend)	Render: 26.12.0 / Local: 24.17.0	Runtime environment for backend API gateway, indexer, and keeper bot.
Express.js	^4.19.2	REST API server framework for officer authentication, nonces, and gas relays.
node:sqlite	Native Node.js built-in	Zero-dependency embedded database for read caching and off-chain indexing.
jsonwebtoken	^9.0.2	Issues and verifies scoped officer session tokens with PSU tenant claims.
bcryptjs	^2.4.3	Salted hashing of officer authentication passwords in SQLite database.
@pinata/sdk	^2.1.0	Pushes tender specification files and KYC documents to the IPFS network.
React	^18.3.1	Frontend library for the single-page user interface.
Vite	^5.4.1	Build tool and frontend development server.
Tailwind CSS	^3.4.10	Utility-first CSS framework adhering to GIGW design standards.
Lucide React	^0.435.0	Accessible UI iconography across tender stages.
Canvas Confetti	^1.9.4	User interface celebration animations

		upon successful tender award.
LibreOffice Headless	7.6.x (Linux)	Generates identical PDF documents from built DOCX submissions.

8. AI Models Used

During this project, I used several large language model assistants as programming aids. The code, test suites, and configuration scripts in the repository were generated by these tools based on my technical instructions and specifications:

- **Claude 3.5 Sonnet:** Used for initial scaffolding of the Solidity smart contract, UUPS proxy setup, and Hardhat test assertions.
- **Claude 3.7 Sonnet:** Used for structuring backend event indexer fallback logic, SQLite schema setup, and test debugging.
- **GPT-4o:** Used for generating React component layouts, GIGW accessibility styling, and Tailwind design adjustments.
- **DeepSeek-R1:** Used for analyzing EVM gas optimization trade-offs and verifying edge cases in dispute timeout logic.

Student Responsibility Statement

I used AI tools as assistive programming aids for code scaffolding, documentation drafting, and test generation under my direction. I reviewed all generated smart contract code, backend routes, and test suites, verified them against the procurement rules, ran the test suites locally, and deployed the contracts and web services to their respective platforms. I take full responsibility for the design, architectural decisions, and final implementation presented in this report.

9. Detailed Architecture Diagram

9.1 System Architecture

Figure 1 illustrates the multi-tier system architecture of NIRVAAN, showing the relationship between client browsers, Vercel CDN hosting, the Express backend and SQLite indexer on Render, Pinata IPFS, and the Ethereum Sepolia blockchain.

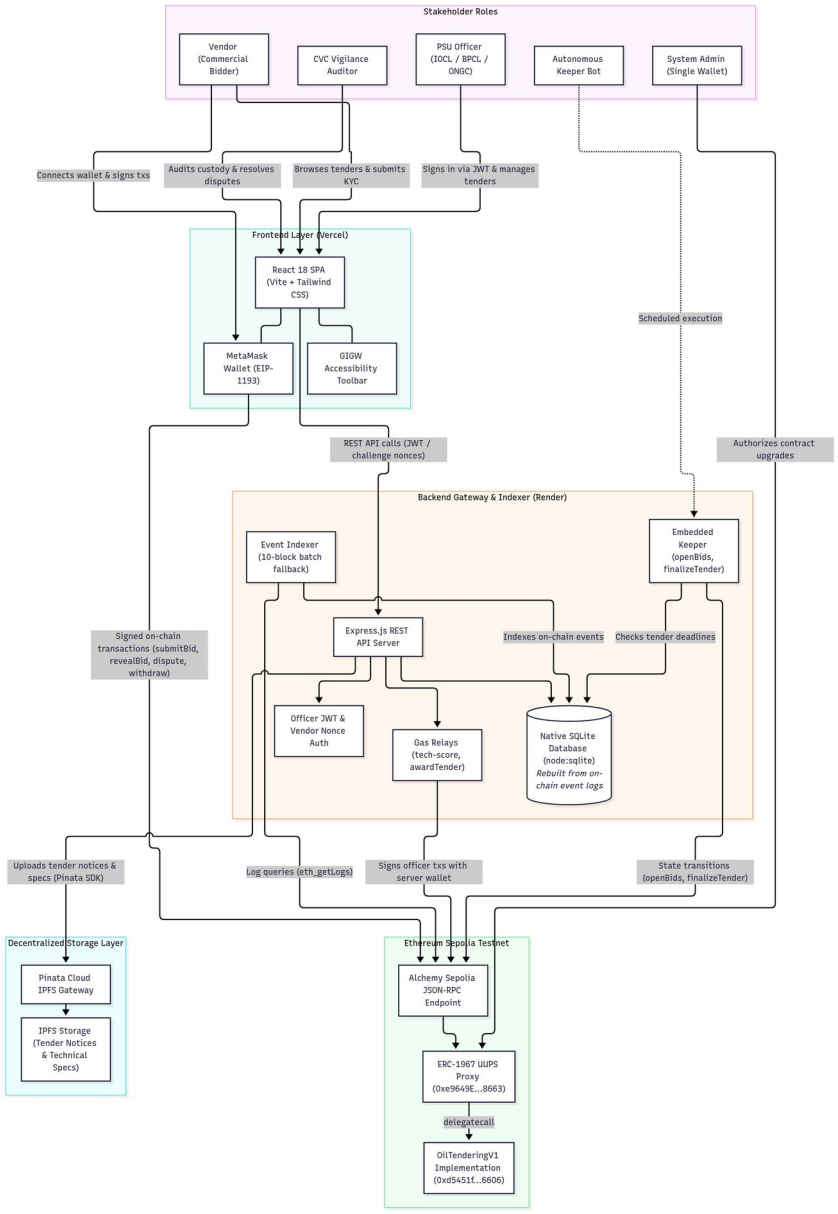


Figure 1: System Architecture of the NIRVAAN Platform

In this architecture, users interact with the React single-page frontend hosted on Vercel. Commercial bidders sign on-chain transactions directly using MetaMask, while officers authenticate through scoped JWTs with backend gas relays. The backend runs an event indexer that mirrors on-chain state into local SQLite, an embedded keeper that triggers time-dependent transitions, and pins documents to IPFS via Pinata.

9.2 Tender State Machine Model

Figure 2 illustrates the finite state machine encoded in the smart contract enum TenderStatus, covering all seven contract states: Created, UnderEvaluation, Awarded, Disputed, DisputeResolved, Finalized, and Cancelled.

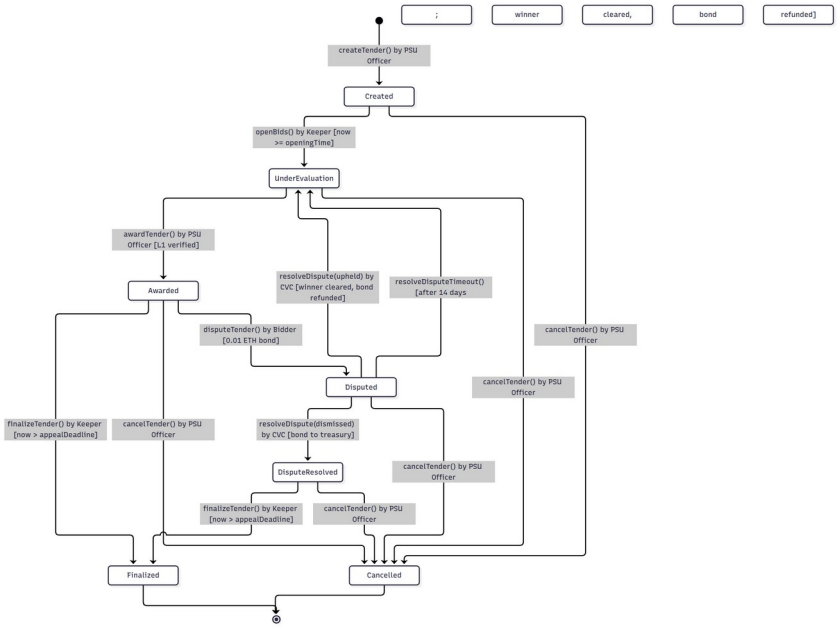


Figure 2: Tender Lifecycle Finite State Machine Model

Tenders begin in Created and move to UnderEvaluation once keeper triggers openBids() after the submission deadline. Following reveal and scoring, the officer awards the tender to the lowest responsive bidder, initiating a 7-day standstill. If appealed, an upheld dispute or a 14-day timeout clears the winner and returns the tender to UnderEvaluation; a dismissed dispute transitions status to DisputeResolved and forfeits the bond to the treasury. FinalizeTender() moves eligible tenders to Finalized, and cancelTender() cancels any active tender while refunding EMDs.

9.3 Bid Lifecycle Sequence Model

Figure 3 depicts the chronological sequence of interactions across all seven lifecycle phases between the Vendor, PSU Officer, CVC Auditor, Express Backend, Autonomous Keeper, and the OilTenderingV1 smart contract.

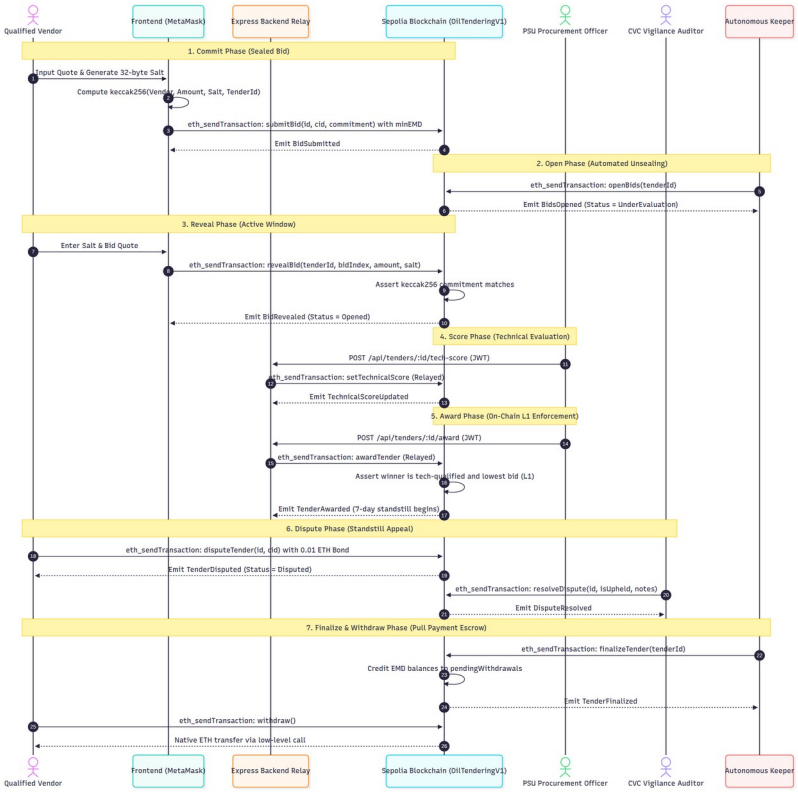


Figure 3: End-to-End Bid Lifecycle Sequence Model

The sequence starts with the vendor computing a client-salted hash and submitting a sealed bid with EMD. After keeper unsealing and vendor reveal, the officer scores proposals and executes the award on-chain. During the standstill window, disputes can be filed and adjudicated before the keeper finalizes the tender, enabling pull-payment withdrawals.

9.4 Data Partitioning and ER Model

Figure 4 models the data partitioning strategy between on-chain storage, the off-chain SQLite database, and decentralized IPFS storage.

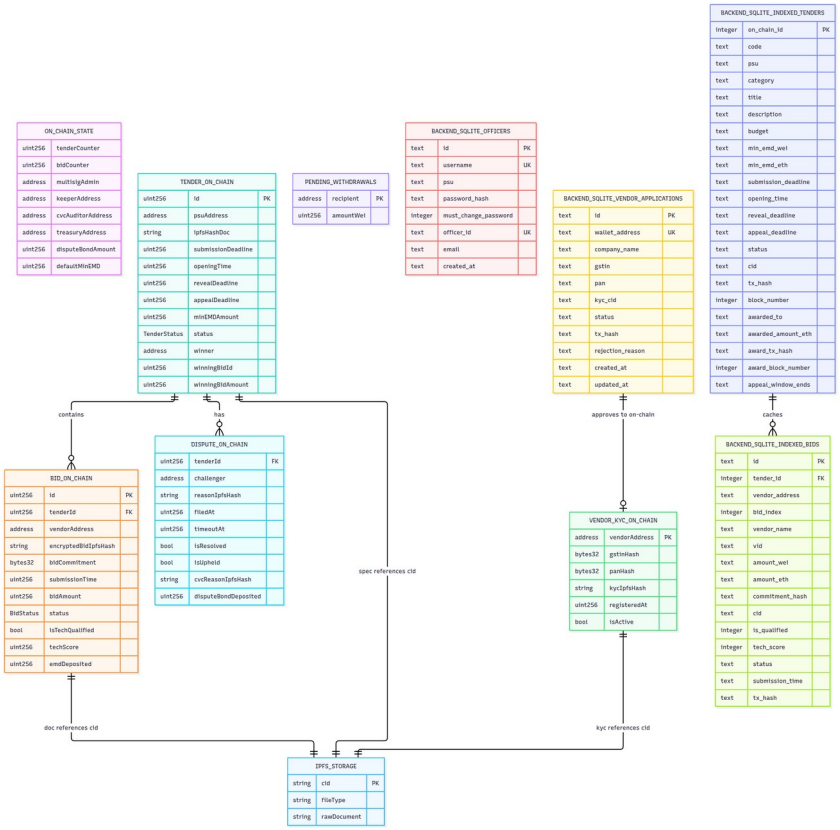


Figure 4: Data Partitioning and Entity-Relationship Model

The smart contract holds core state including tender metadata, bid commitments, dispute records, and pull-payment balances. SQLite acts as an off-chain query cache rebuilt from on-chain event logs on server boot, while IPFS stores large tender specifications and vendor qualification filings referenced by their Content Identifiers (CIDs).

10. Components of Blockchain and Explanation of the Smart Contract Used

10.1 Blockchain Primitives and UUPS Proxy Pattern

The core protocol is built on the Ethereum Virtual Machine (EVM) and utilizes several standard blockchain primitives [1], [2]:

- **Keccak-256 Cryptographic Hashing:** Used for computing sealed bid commitments keccak256(vendor, amount, salt, tenderId) and hashing corporate GSTIN/PAN numbers for vendor privacy.
- **Universal Upgradeable Proxy Standard (UUPS):** Implemented using OpenZeppelin's ERC1967Proxy and UUPSUpgradeable pattern [7], [8]. The proxy delegates execution calls to the implementation contract via delegatecall while maintaining persistent state storage.

- **Pull-Payment Escrow Pattern:** To prevent re-entrancy attacks during ETH transfers, the contract uses OpenZeppelin's ReentrancyGuardUpgradeable and adopts a pull-payment withdrawal pattern where balances are credited to a mapping before withdrawal.

10.2 Data Structures and Functions Reference

The implementation contract OilTenderingV1 defines four primary data structures in Solidity storage:

```
struct Tender {
    uint256 id;
    address psuAddress;
    string ipfsHashDoc;
    uint256 submissionDeadline;
    uint256 openingTime;
    uint256 revealDeadline;
    uint256 appealDeadline;
    uint256 minEMDAmount;
    TenderStatus status;
    address winner;
    uint256 winningBidId;
    uint256 winningBidAmount;
}

struct Bid {
    uint256 id;
    uint256 tenderId;
    address vendorAddress;
    string encryptedBidIpfsHash;
    bytes32 bidCommitment;
    uint256 submissionTime;
    uint256 bidAmount;
    BidStatus status;
    bool isTechQualified;
    uint256 techScore;
    uint256 emdDeposited;
}

struct Dispute {
    uint256 tenderId;
    address challenger;
    string reasonIpfsHash;
    uint256 filedAt;
    uint256 timeoutAt;
    bool isResolved;
    bool isUpheld;
    string cvcReasonIpfsHash;
    uint256 disputeBondDeposited;
}

struct VendorKYC {
    bytes32 gstinHash;
    bytes32 panHash;
    string kycIpfsHash;
```

```
    uint256 registeredAt;
    bool isActive;
}
```

Table 5 details the complete public and external function interface of OilTenderingV1.sol, verifying access control guards and core invariants.

Table 5: Smart Contract External and Public Functions Reference

Function Signature	Access Guard	Payable	Purpose and Core Invariants Enforced
initialize(admin, keeper, cvc, treasury, bond, emd)	External (Initializer)	No	Initializes proxy storage. Configures admin, keeper, CVC auditor, treasury, dispute bond, and default EMD.
registerPSU(psuAddress)	onlyAdmin	No	Accredits an institutional PSU procurement address to create and manage tenders.
registerVendor(vendor, gstinHash, panHash, kycCid)	onlyAdmin	No	Registers a vendor address with hashed tax credentials and IPFS qualification documents.
createTender(ipfsDoc, subDead, openTime, revDead, minEMD)	onlyPSU	No	Publishes a new tender. Enforces subDead > now, openTime >= subDead, revDead > openTime.
cancelTender(tenderId, reason)	onlyPSU	No	Cancels tender prior to finalization and refunds all deposited EMDs inline to pendingWithdrawals.
submitBid(tenderId, encryptedDoc, commitment)	onlyVendor	Yes	Accepts sealed bid commitment. Enforces deadline, minimum EMD, and blocks duplicate bids per vendor.
openBids(tenderId)	onlyKeeper	No	Transitions tender to UnderEvaluation once block.timestamp >= openingTime.
revealBid(tenderId, bidIndex, bidAmount, salt)	onlyVendor	No	Verifies salt and quote match stored commitment. Updates bid status to Opened.
setTechnicalScore(tenderId, bidIndex, qualified, score)	onlyPSU	No	Records technical score (0-100) and qualification flag for

			opened proposals.
awardTender(tenderId, winner, bidId, bidAmount)	onlyPSU	No	Enforces lowest qualified bid (L1) on-chain. Marks unrevealed bids Rejected and starts 7-day standstill.
disputeTender(tenderId, reasonIpfs)	onlyVendor	Yes	Accepts standstill challenge with required dispute bond (0.01 ETH). Sets status to Disputed.
resolveDispute(tenderId, upheld, cvcReasonIpfs)	onlyCVC	No	If upheld: clears winner, refunds bond, returns to UnderEvaluation. If dismissed: status to DisputeResolved, bond to treasury.
resolveDisputeTimeout(tenderId)	Public	No	If CVC does not rule within 14 days, clears winner, refunds bond, and returns tender to UnderEvaluation.
finalizeTender(tenderId)	onlyKeeper	No	Callable after standstill window expires. Invokes _creditRefunds() for non-rejected bids.
withdraw()	External (nonReentrant)	No	Transfers pendingWithdrawals balance to msg.sender using low-level call.
setKeeperAddress(newKeeper)	onlyAdmin	No	Updates the authorized keeper wallet address.
setCVCAuditorAddress(newCVC)	onlyAdmin	No	Updates the authorized Central Vigilance Commission auditor wallet address.

10.3 Security Patterns and Measured Gas Profile

Table 6 reports the exact gas consumption measured from transaction execution receipts on the Hardhat local network. No values are rounded or estimated.

Table 6: Smart Contract Gas Consumption Profile Measured on Hardhat EVM

Smart Contract Transaction	Execution Context	Measured Gas (Units)
Implementation Deployment	Initial contract compilation and	3,971,912

	bytecode deployment	
ERC1967Proxy Deployment	UUPS Proxy deployment pointing to implementation	313,204
registerPSU	Admin registration of issuing PSU address	52,451
registerVendor	Admin onboarding with GSTIN/PAN hashes	166,350
createTender	Publishing tender notice and parameters	264,487
submitBid (First Bid)	First sealed commitment and EMD storage allocation	270,488
submitBid (Subsequent)	Subsequent bidder commitment storage	236,288
openBids	Keeper state transition to UnderEvaluation	56,274
revealBid (First Reveal)	Commitment hash check and quote unsealing	87,187
revealBid (Subsequent)	Subsequent bidder quote unsealing	87,199
setTechnicalScore	Officer technical compliance scoring	65,407
awardTender	On-chain L1 validation loop and winner assignment	132,698
disputeTender	Standstill challenge and dispute bond deposit	203,739
resolveDispute (Dismissed)	CVC ruling with bond transfer to treasury	110,642
finalizeTender	Keeper finalization and refund balance credits	95,452
withdraw	Pull-payment transfer to caller wallet	34,948
cancelTender	Tender cancellation with inline bidder refunds	84,511

10.4 Known Limitations and Prototype Scope

Review of the smart contract and backend architecture reveals several technical limitations in this prototype:

- **Unrevealed Bid EMD Retention:** Under awardTender(), any unrevealed bids remain in BidStatus.Submitted and are marked Rejected. In _creditRefunds(), only non-rejected bids receive refunds. Consequently, the EMD of unrevealed bids is not redistributed or transferred to the treasury, remaining locked in the contract balance.
- **Public Mempool Exposure During Reveal:** While bid amounts are concealed during the submission window, reveal transactions publish plaintext quotes to public mempools before confirmation. On a public network, front-running by competing revealers is mitigated by the fixed reveal deadline, but transaction ordering dependencies remain.
- **Centralized Worker and Keeper Deployment:** In this prototype, the backend API server, event indexer, and keeper cron run on a single Render instance. If this instance experiences downtime, scheduled state transitions (such as openBids) pause until the service restarts.
- **Single Admin Private Key:** The admin role is controlled by a single private key rather than a multi-signature consortium wallet (such as Gnosis Safe), which would be required for a production multi-PSU consortium.

11. Sample Screenshots

Below are screenshots of the running platform components captured from the live deployment environments.

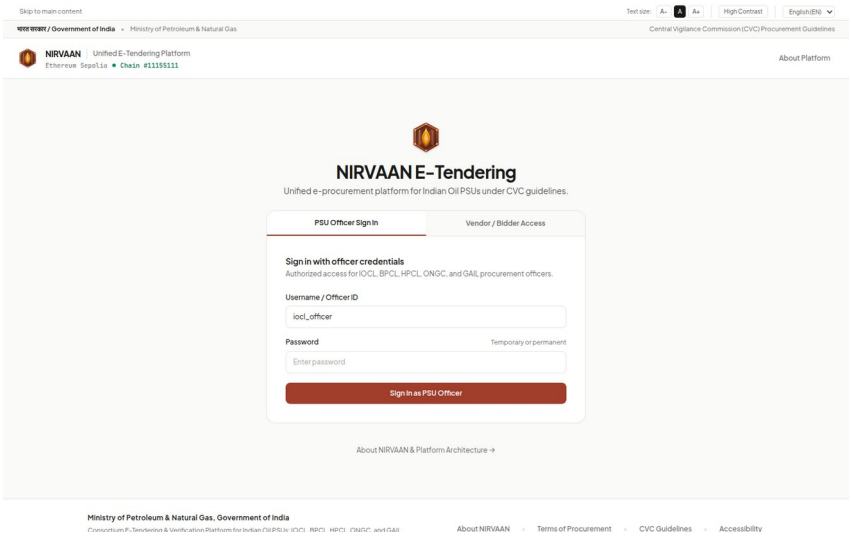


Figure 5: PSU Officer Authentication Portal (Live at https://nirvaan-flame.vercel.app/)

Figure 5 shows the authentication portal for PSU officers from IOCL, BPCL, HPCL, ONGC, and GAIL. Officers enter their assigned credentials to receive a scoped JSON Web Token (JWT) that enforces tenant isolation across administrative routes.



Figure 6: Backend Microservice Runtime Health and Diagnostics Endpoint (Live on Render)

Figure 6 displays the runtime health endpoint (/api/health) on Render. The JSON payload verifies that the backend relay is operational, the event indexer is synchronized with Sepolia block height with zero block lag, and the autonomous keeper is running.

12. Conclusion

In this capstone project, I designed, integrated, and verified NIRVAAN, an open e-tendering platform prototype for Indian oil and gas PSUs. The smart contract, backend services, and user interface were scaffolded using AI coding tools based on my architectural specifications, after which I reviewed, debugged, tested, and deployed the implementation. By combining client-side salted commitments, on-chain L1 award verification, an escrow pull-payment model, and an independent CVC standstill arbitration window, the prototype demonstrates how core procurement rules can be enforced deterministically.

Table 7 summarizes the automated verification test suites executed across all platform components before deployment.

Table 7: Test Suite Execution Summary

Test Suite Level	Scope and Execution Harness	Passing / Total	Pass Rate
Smart Contract Unit Tests	Hardhat EVM tests for proxy initialization, access guards, commit-reveal hashing, L1 awards, and dispute timeouts.	13 / 13	100%
Backend Integration Tests	Node.js test runner for SQLite indexer fallback, Alchemy 10-block batch recovery, and KYC	24 / 24	100%

	application workflows.		
End-to-End Smoke Tests	Full lifecycle verification across tender creation, vendor onboarding, bid submission, scoring, and award.	36 / 36	100%
Repository Secrets Audit	Automated regex scan across all 127 tracked repository files for exposed private keys and API tokens.	127 / 127 clean	100%

Lessons Learned: Working with smart contracts highlighted the importance of clear state machine definitions and minimal on-chain loops to keep gas costs manageable. Writing automated tests for keeper state transitions and error fallback in the event indexer was critical for reliable synchronization between Sepolia and the off-chain cache.

Future Work: For a production deployment, future iterations should transition the single administrator wallet to a multi-signature consortium wallet (e.g. Gnosis Safe), implement decentralized keeper networks (e.g. Chainlink Automation), add zero-knowledge proofs for technical qualification scoring, and address unrevealed EMD redistribution.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. Available: https://bitcoin.org/bitcoin.pdf

[2] V. Buterin, "A Next-Generation Smart Contract and Decentralized Application Platform," Ethereum White Paper, 2014. Available: https://ethereum.org/en/whitepaper/

[3] Central Vigilance Commission, Government of India, "Vigilance Manual," 2021. Available: https://www.cvc.gov.in/

[4] Ministry of Finance, Department of Expenditure, Government of India, "General Financial Rules (GFR)," Rule 170 (Bid Security), 2017. Available: https://doe.gov.in/order-rules/general-financial-rules-2017

[5] K. Wüst and A. Gervais, "Do you need a Blockchain?," in Proceedings of the 2018 Crypto Valley Conference on Blockchain Technology (CVCBT), Zug, Switzerland, 2018, pp. 45–54. doi: 10.1109/CVCBT.2018.00011.

[6] OpenZeppelin, "OpenZeppelin Contracts (v5.0.0)," 2023. Available: https://github.com/OpenZeppelin/openzeppelin-contracts

[7] Ethereum Foundation, "ERC-1967: Standard Proxy Storage Slots," EIP-1967, 2019. Available: https://eips.ethereum.org/EIPS/eip-1967

[8] Ethereum Foundation, "ERC-1822: Universal Upgradeable Proxy Standard (UUPS)," EIP-1822, 2019. Available: https://eips.ethereum.org/EIPS/eip-1822